

Stage 2: Build a Reproducible Pipeline

Reproducibility

Task

Convert the exploratory script or notebook from Stage 1 into a fully modular, production shaped data pipeline a structural re-architecture, not a bug fix pass. The pipeline must run its cleaning transformations as a distributed Spark job, persist its cleaned output to a local database table, and announce its own completion as an event that a separate consumer process picks up and acts on.

Deliverables & Architectural Requirements

1. Config Driven Design

Eliminate hardcoded logic. Dynamic elements merchant lookup tables, date format specs, currency/FX metadata, trap pair rules must live in configuration files or in database tables, not in code.

2. Distributed Processing (Spark)

The cleaning transformations run as Spark DataFrame operations, not row wise pandas. Type coercion (dates, amounts, currencies) happens inside Spark with explicit casts, so unparseable rows can be counted and logged rather than silently zeroed or dropped. A local Spark session is sufficient no cluster required.

3. Persistence Local Database Table

Cleaned output writes to a local database table (Postgres, SQLite, or equivalent), not just a file. The write must be idempotent: rerunning the pipeline on the same input must not duplicate rows an upsert on a natural or derived key, not a blind INSERT.

4. Event Emission

After a successful write, the pipeline emits an event to Kafka announcing what happened at minimum a run id, row counts (read / cleaned / dropped / flagged), and a timestamp. The candidate must explicitly use Kafka as the event emission mechanism. What this tests: can the candidate draw a real boundary between “the pipeline finished” and “something else needs to know”.

5. Event Consumption

A separate consumer process listens for that event and acts on it at minimum, logging receipt and validating the payload; ideally, kicking off a downstream step (e.g. a placeholder hook for feature extraction). The consumer must be resilient to replay: if the same event is delivered twice (a crash and retry, a redelivered message), processing it twice must not corrupt state. This is the same idempotency discipline as requirement 3, now enforced

across a process boundary instead of within one script.

6. **End to End Idempotency**

Running the full chain clean → persist emit consume twice on identical input must leave the system in the identical state it would reach after one run. This is stricter than requirement 3 alone: it has to hold through the event and the consumer, not just the database write.

7. **Auditability & Logging**

An explicit audit trail of what was coerced, dropped, or flagged, and why at the cleaning stage, at the database write, and at the consumer. No silent cleaning anywhere in the chain.

8. **Automated Testing**

A test suite covering: the trap pair cases, the FX reconciliation formula, the database upsert's duplicate run behavior, and the consumer's replay behavior (same event delivered twice → same end state).